



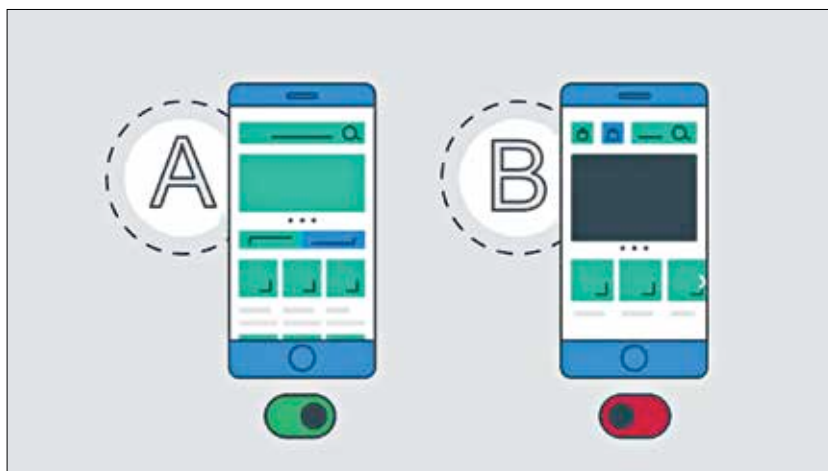
Zoom infrastructure

To adapt to surge in demand, Zoom has picked Oracle Cloud Infrastructure after a quick selection process. <https://digit.in/may20-11>



What makes a great chatbot?

While it is important to have laser focus on customers, continuously improving a chatbot decides its success. <https://digit.in/may20-12>



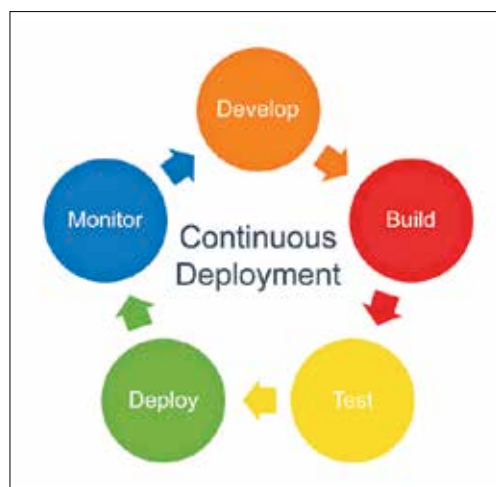
A/B Testing

tickets. Say your usual process of obtaining a ticket on your website requires 11 steps and you want to check what kind of conversion improvement can be accomplished by reducing the number of steps required to make a purchase. So you'd run an A/B test to see exactly what kind of difference it brings to your application.

4. Continuous Deployment

Continuous deployment is a software development strategy where you gradually introduce new features to your application rather than making massive changes that could drastically alter the user experience. This is used by practically all major software and technology companies world wide to push new features to their users. CI/CD or Continuous Integration and Continuous Delivery would be how you incorporate new features into your existing application only to turn it on at the drop of a hat. Let's say you're developing an application for Android and iOS. Normally, if you want to introduce a new change to your entire user base you would have to wait till all your users downloaded the latest update. Otherwise, you would be running two versions of the code on your server for the older code and the newer code. Using feature flags, you can

push the new feature to all your users over a longer period of time so that all your users would have downloaded compatible versions of the app. Then all you need to do is switch the feature flag and your entire code base will be moved to the new branch.



TYPES OF FEATURE FLAGS

While feature flags can be implemented in hundreds of different ways here are some common ways that you'd want to explore implementing the same.

1. **Release toggles:** Think of these like simple on/off switches for enabling features. These are mostly not related to existing functionality but rather with new additions to the application. Once the new feature is released, this toggle can be deleted.

2. **Operational toggles:** These are also short-lived toggles for switching out certain functions, algorithms and modules that are to be deprecated. Once the task has been performed, these toggles can be removed.

3. **Experimental Toggles:** When you want to perform A/B testing, you'd want to implement such toggles. These will divert a section of your users through a different section of the code as part of your A/B testing exercise. These kinds of toggles are long-term since you'd want to revisit such functions to perform A/B testing again and again.

4. **Permission Toggles:** If you want to segregate users then you can implement permission toggles. Let's say you have certain users who've paid for accessing a larger feature set whereas the rest of the users are free-users. Permission toggles would allow you to provide a greater user experience to your paid users.

WHEN AND HOW TO USE FEATURE FLAGS

Feature flags essentially create a new branch in how the code functions. So you can imagine what would happen if you went overboard with throwing in feature flags at every single step of the way. A good example of what could end up happening if you use too many feature flags is to look at the Android ecosystem. With so many different manufacturers, creating multiple devices, and that too with hundreds of different variations of hardware have created a massively fragmented ecosystem where every user experience is different from the next person. Pushing new updates to such an ecosystem has become totally impossible. One way Google is tackling this is with Project Treble. Project Treble separates the firmware into two sections, one which is maintained by Google and the other maintained by the vendor. Key OS modules are retained in the section maintained by Google whereas all